Universidad Tecnológica de Montemorelos

INGENIERÍA EN SOFTWARE · DSM4

INVESTIGACIÓN Y REFLEXIÓN

Actividad integradora: HTML5, CSS3, Usabilidad y Diseño Responsivo

Materia	Desarrollo de aplicaciones WEB
Docente	Edgar Gonzales
Alumno	Joel Humberto Solis Ramírez
Matrícula	27437
Carrera	Ingeniería en Software
Grupo	DSM4

Montemorelos, N.L. · 26/10/26

Contenido

1. Parte I. HTML5
2. Parte II. DOM y diagrama
3. Parte III. Etiquetas estructurales
4. Parte IV. Etiquetas de presentación y ejemplos
5. Parte V. Metadatos
6. Parte VI. Formularios y validación
7. Parte VII. Usabilidad y accesibilidad
8. Parte VIII. Análisis de un sitio web conocido
9. Parte IX. CSS y CSS3
0. Parte X. Reglas y selectores CSS
1. Parte XI. Propiedades, modelo de caja, Flexbox, diseño responsivo y frameworks
2. Reflexión final: 15 respuestas sobre el proyecto
3. Bibliografía

> **Propósito.** Integrar los conceptos que permiten construir interfaces web semánticas, comprensibles, accesibles, adaptables y mantenibles. Los ejemplos son originales y están planteados para un sitio de una empresa tecnológica desarrollado con HTML5, CSS3 y Bootstrap.

Parte I. HTML5

1. ¿Qué es HTML5 y cuál es su función en una página web?

> **HTML5** es la evolución moderna del lenguaje de marcado HTML. Su función es describir la estructura y el significado del contenido que interpreta un navegador: títulos, párrafos, navegación, imágenes, formularios, tablas, audio, video y regiones de una página. No es un lenguaje de programación porque no define por sí solo algoritmos; organiza información mediante elementos y atributos.
>
> HTML forma una de las tres capas básicas del desarrollo web: HTML aporta estructura y semántica; CSS controla presentación y distribución; JavaScript agrega comportamiento e interacción. Separarlas favorece el mantenimiento y permite que navegadores, buscadores y tecnologías de asistencia comprendan mejor el documento.

2. ¿Qué aportó HTML5 frente a versiones anteriores?

> HTML5 simplificó la declaración del documento con `<!doctype html>`, incorporó elementos semánticos como `header`, `nav`, `main`, `section`, `article`, `aside` y `footer`, y normalizó contenido multimedia con `audio`, `video` y `canvas`. También amplió los formularios con tipos como `email`, `date`, `number` y atributos de validación como `required`, `pattern`, `min` y `max`.
>
> **Diferencia entre HTML y HTML5:** HTML es el nombre general de la familia del lenguaje de marcado; HTML5 identifica su modernización, que incorporó semántica, multimedia y formularios mejorados. Frente a versiones antiguas, HTML5 evita varias etiquetas puramente visuales, simplifica el doctype y ofrece elementos nativos que antes dependían de complementos o soluciones externas.
>
> Su enfoque semántico reduce el abuso de contenedores genéricos, mejora la accesibilidad y comunica el propósito de cada bloque. En la práctica, HTML se mantiene como un estándar vivo: recibe correcciones y mejoras graduales en lugar de depender únicamente de grandes versiones aisladas.

3. ¿Cuál es la estructura mínima correcta de un documento HTML5?

```
<!doctype html>
<html lang="es-MX">
<head>
 <meta charset="utf-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Empresa tecnológica</title>
</head>
<body>
 <main>
  <h1>Soluciones digitales</h1>
 </main>
</body>
</html>
```

> La función de cada parte es la siguiente:
>
> | Parte | Función |
> | --- | --- |
> | `<!doctype html>` | Declara que el documento usa HTML moderno y hace que el navegador emplee su modo estándar. No es una etiqueta visible. |
> | `<html>` | Es el elemento raíz que contiene todo el documento. Su atributo `lang` comunica el idioma principal. |
> | `<head>` | Reúne metadatos y recursos no presentados como contenido principal: título, codificación, viewport, descripción y enlaces a estilos. |
> | `<body>` | Contiene la información e interfaz que el usuario percibe: textos, imágenes, navegación, secciones, formularios y scripts asociados. |
>
> El encabezado principal `h1` expresa el tema central de la página.

Parte II. DOM y diagrama

1. ¿Qué es el DOM?

> El **Document Object Model (DOM)** es una representación en memoria del documento como un árbol de objetos. Cada etiqueta, texto y atributo se convierte en un nodo relacionado con nodos padre, hijos y hermanos. Esta interfaz permite que JavaScript consulte o modifique contenido, atributos, clases y eventos sin reescribir todo el archivo.
>
> HTML es el texto fuente; el DOM es el modelo que el navegador construye después de analizarlo. Si JavaScript ejecuta `document.querySelector("#contacto")`, busca en ese árbol el elemento cuyo identificador sea `contacto`. Una modificación del DOM se refleja visualmente en la página renderizada, aunque no cambia automáticamente el archivo HTML guardado en el servidor.

2. ¿Cómo se organiza el árbol DOM? Incluye un diagrama.

> El nodo raíz es `document`; normalmente contiene `html`, y este divide el documento en `head` y `body`. A partir de ahí se forman relaciones jerárquicas.
>
> ```
> document
> □□□ HTML
>   □□□ HEAD
>   □   □□□ TITLE
>   □       □□□ "Mi página"
>   □□□ BODY
>     □□□ H1
>     □   □□□ "Mi sitio Web"
>     □□□ P
>       □□□ "Bienvenido"
> ```
>
> En el documento analizado, `head` y `body` son hijos de `html`. Dentro de `head` se encuentra `title`; dentro de `body`, `h1` y `p` son hermanos. Los textos entre etiquetas también son nodos.

3. ¿Qué es un nodo, un elemento y un atributo?

> - **Nodo:** unidad del árbol DOM. Puede representar el documento, un elemento, un texto, un comentario u otro tipo de dato.
> - **Elemento:** nodo creado a partir de una etiqueta, como `p`, `img` o `form`. Puede contener atributos y nodos hijos.
> - **Atributo:** dato asociado a un elemento que configura o describe su comportamiento, por ejemplo `href`, `src`, `id`, `class` o `alt`.
>
> El navegador analiza el texto HTML y construye esta jerarquía en memoria. Después combina el DOM con CSS para representarlo visualmente y expone métodos para que JavaScript lo consulte.

4. ¿Cómo puede JavaScript modificar el DOM?

> Sirve para responder a acciones del usuario y actualizar la interfaz: abrir un menú, validar un formulario, filtrar tarjetas, mostrar un mensaje o cambiar un tema visual. Debe hacerse con

> moderación: actualizaciones pequeñas y deliberadas evitan problemas de rendimiento, y cualquier cambio debe conservar nombre, foco y estado accesibles.

Parte III. Etiquetas estructurales

1. ¿Qué son las etiquetas estructurales o semánticas?

> Son elementos cuyo nombre explica la función de la región que contienen. Dan un esquema lógico al documento y crean puntos de referencia útiles para navegación con teclado, lectores de pantalla, motores de búsqueda y mantenimiento del código.

Etiqueta	Función	Uso recomendado
`<header>`	Contenido introductorio o encabezado.	Marca, título y controles iniciales de una página o sección.
`<nav>`	Bloque de navegación principal.	Menú del sitio, índice o paginación relevante.
`<main>`	Contenido principal único.	Debe existir como máximo uno visible por documento.
`<section>`	Agrupación temática.	Debe contar normalmente con un encabezado que la identifique.
`<article>`	Contenido autónomo y reutilizable.	Noticia, publicación, tarjeta detallada o entrada.
`<aside>`	Contenido complementario.	Datos relacionados, glosario o llamada lateral.
`<footer>`	Pie de página o sección.	Autoría, contacto, enlaces legales o cierre.
`<address>`	Información de contacto.	Datos de contacto del autor o entidad vinculada.

2. ¿Cuándo usar `div` en lugar de una etiqueta semántica?

> `div` es un contenedor genérico válido cuando no existe un elemento semántico adecuado, por ejemplo, para agrupar piezas únicamente por distribución visual. No debe elegirse por costumbre si el bloque representa claramente navegación, contenido principal, artículo o pie de página. La semántica describe el contenido; las clases CSS describen su apariencia.

Ejemplo integrado:

```
<header>
 <nav aria-label="Navegación principal">…</nav>
</header>
<main>
 <section aria-labelledby="servicios-titulo">
  <h2 id="servicios-titulo">Servicios</h2>
  <article><h3>Desarrollo web</h3><p>…</p></article>
 </section>
</main>
<footer><address>contacto@empresa.example</address></footer>
```

Parte IV. Etiquetas de presentación y ejemplos

1. ¿Qué elementos presentan contenido textual o multimedia?

> Se les suele llamar “etiquetas de presentación”, aunque en HTML moderno conviene pensar en **estructura y significado**, dejando color, tamaño y posición a CSS. Los elementos más comunes son:

Elemento	Propósito	Ejemplo
`h1` – `h6`	Jerarquía de encabezados.	`<h2>Nuestros servicios</h2>`
`p`	Párrafo.	`<p>Creamos productos digitales.</p>`
`strong`	Importancia fuerte.	`<strong>Entrega segura</strong>`
`em`	Énfasis que cambia el sentido.	`<em>Primero</em> diseñamos.`
`mark`	Texto relevante o resaltado.	`<mark>Nueva versión</mark>`
`small`	Comentario secundario o legal.	`<small>Aplican restricciones.</small>`
`del`	Contenido eliminado o que dejó de ser vigente.	`<del>$1,200</del>`
`ins`	Contenido añadido posteriormente.	`<ins>Nueva condición</ins>`
`sub`	Subíndice tipográfico con significado.	`H<sub>2</sub>O`
`sup`	Superíndice tipográfico con significado.	`m<sup>2</sup>`
`abbr`	Abreviatura explicada.	`<abbr title="HyperText Markup Language">HTML</abbr>`
`blockquote` / `q`	Cita en bloque o breve.	`<q>Diseñar es resolver.</q>`
`code` / `pre`	Código y texto preformateado.	`<code>display: flex;</code>`
`ul` / `ol` / `li`	Listas sin orden o numeradas.	`<ul><li>Diseño</li></ul>`
`a`	Hipervínculo.	`<a href="#contacto">Contacto</a>`
`img` / `figure` / `figcaption`	Imagen, unidad ilustrada y descripción.	`<img src="equipo.webp" alt="Equipo colaborando">`
`audio` / `video`	Multimedia nativa.	`<video controls>…</video>`

2. ¿Por qué ya no se recomiendan etiquetas como `font`, `center` o `big`?

> Porque mezclan contenido con apariencia, están obsoletas y dificultan consistencia, accesibilidad y mantenimiento. En lugar de `<center>` se usa una regla como `text-align: center`; en lugar de `<font color="blue">`, una clase CSS. Elementos como `strong` y `em` sí son apropiados cuando expresan significado, no solo estilo visual.

Parte V. Metadatos

1. ¿Qué son los metadatos y dónde se colocan?

> Son datos que describen el documento y orientan al navegador, buscadores y plataformas, aunque normalmente no aparecen como contenido principal. Se colocan dentro de `head`. No sustituyen un contenido bien estructurado, pero ayudan a interpretar idioma, codificación, escala, autoría y resumen.

2. ¿Cuáles son los metadatos esenciales y qué función cumplen?

> - `<meta charset="utf-8">`: selecciona UTF-8 como codificación, permitiendo representar correctamente acentos, eñes, símbolos y caracteres de múltiples idiomas.
> - `<meta name="viewport" content="width=device-width, initial-scale=1.0">`: controla el área de visualización en dispositivos móviles. `width=device-width` iguala el ancho lógico de la página al ancho de la pantalla; `initial-scale=1.0` establece un nivel inicial de zoom de 100 %. Es importante porque, sin esta configuración, un teléfono puede simular una ventana de escritorio, mostrar todo demasiado pequeño e impedir que las media queries correspondan al espacio real.
> - `<title>`: identifica la pestaña, el historial y el título frecuente en resultados de búsqueda.
> - `description`: resumen conciso y específico de la página.
> - `author`: autor responsable del documento cuando sea pertinente.
> - `link rel="canonical"`: señala la dirección preferida cuando el mismo contenido tiene URL alternativas.
> - `link rel="icon"`: declara el ícono del sitio.
> - Metadatos sociales, como Open Graph, ayudan a controlar la vista previa al compartir; deben reflejar el contenido real.
>
> ```
> <head>
>  <meta charset="utf-8">
>  <meta name="viewport" content="width=device-width, initial-scale=1">
>  <meta name="description" content="Servicios de desarrollo web y nube.">
>  <meta name="author" content="Equipo NovaTech">
>  <title>NovaTech | Soluciones digitales</title>
>  <link rel="canonical" href="https://www.example.com/">
> </head>
> ```

Parte VI. Formulario y validación

1. ¿Qué es un formulario HTML y cuáles son sus elementos principales?

> Un formulario recopila información para procesarla. `form` agrupa controles; `label` nombra cada campo; `input`, `select` y `textarea` capturan datos; `fieldset` y `legend` agrupan opciones relacionadas; y `button` ejecuta una acción. Para un envío real, los controles requieren `name`, y el formulario necesita destino y método definidos por la aplicación.

2. Construye un formulario accesible con validación nativa.

```
<form action="/solicitudes" method="post">
 <div>
  <label for="nombre">Nombre completo</label>
  <input id="nombre" name="nombre" type="text"
       autocomplete="name" minlength="3" maxlength="80" required>
 </div>

 <div>
  <label for="correo">Correo empresarial</label>
  <input id="correo" name="correo" type="email"
       autocomplete="email" aria-describedby="ayuda-correo" required>
  <small id="ayuda-correo">Ejemplo: nombre@empresa.com</small>
 </div>

 <div>
  <label for="telefono">Teléfono a 10 dígitos</label>
  <input id="telefono" name="telefono" type="tel"
       inputmode="numeric" pattern="[0-9]{10}"
       autocomplete="tel" required>
 </div>

 <div>
  <label for="servicio">Servicio de interés</label>
  <select id="servicio" name="servicio" required>
   <option value="">Selecciona una opción</option>
   <option value="web">Desarrollo web</option>
   <option value="nube">Soluciones en la nube</option>
  </select>
 </div>

 <div>
  <label for="mensaje">Describe tu proyecto</label>
  <textarea id="mensaje" name="mensaje" rows="5"
         minlength="20" maxlength="500" required></textarea>
 </div>

 <label>
  <input name="privacidad" type="checkbox" required>
  Acepto el aviso de privacidad
 </label>
 <button type="submit">Enviar solicitud</button>
</form>
```

3. ¿Cómo funciona la validación y qué prácticas deben aplicarse?

El navegador evalúa restricciones declarativas: `required` evita datos vacíos; el tipo `email` comprueba una forma básica; `minlength` / `maxlength` limitan longitud; `min` / `max` delimitan valores; y `pattern` aplica una expresión regular. JavaScript puede complementar la experiencia mediante la API `checkValidity()`, `reportValidity()` y mensajes específicos.

1. Usar una etiqueta visible asociada mediante `for` e `id`; un `placeholder` no reemplaza la etiqueta.
2. Explicar formato y restricciones antes del envío.
3. Mostrar el error junto al campo, indicar cómo corregirlo y mover el foco de forma razonable.
4. No depender solo del color; combinar texto, borde, ícono o estado.
5. Conservar los datos correctos si ocurre un error.
6. Validar otra vez en el servidor: la validación del navegador mejora la experiencia, pero puede omitirse o alterarse y no constituye una barrera de seguridad.
7. Sanear y tratar datos conforme a su contexto de salida, aplicar protección CSRF cuando corresponda y recolectar únicamente la información necesaria.

> **Conclusión de seguridad:** la validación del cliente ofrece retroalimentación rápida; la validación del servidor es la autoridad final para integridad, permisos y seguridad.

4. ¿Qué ventajas tiene utilizar las validaciones de HTML5 antes de JavaScript?

Son declarativas, rápidas y están integradas en el navegador; reducen código repetido, funcionan incluso si un script tarda en cargar y proporcionan controles adecuados para teclado móvil y tecnologías de asistencia. También mantienen reglas visibles en el propio HTML. JavaScript puede mejorar mensajes o flujos complejos, pero no debería reemplazar restricciones simples. Ninguna validación del cliente sustituye la validación obligatoria del servidor.

Parte VII. Usabilidad y accesibilidad

1. ¿Qué es usabilidad?

> Es el grado en que personas específicas pueden utilizar un producto para alcanzar objetivos con efectividad, eficiencia y satisfacción en un contexto determinado. En una web implica saber dónde se está, entender qué ofrece la página, completar tareas sin confusión, prevenir errores y recuperarse de ellos.

2. ¿Qué es accesibilidad web?

> Es la práctica de diseñar y desarrollar sitios que puedan percibirse, operarse, comprenderse y utilizarse de forma robusta por personas con distintas capacidades, dispositivos y contextos. Incluye compatibilidad con teclado, lectores de pantalla, ampliación, alto contraste y preferencias de movimiento. Accesibilidad y usabilidad se complementan: una interfaz puede cumplir técnicamente algunas reglas y aun así ser difícil de entender.

3. ¿Qué características debe tener una interfaz fácil de utilizar y por qué se considera al usuario?

> Debe ser clara, consistente, legible, predecible, rápida de aprender, sencilla de navegar y capaz de prevenir errores o ayudar a corregirlos. Sus controles necesitan etiquetas comprensibles y retroalimentación inmediata. Considerar al usuario permite diseñar para sus objetivos, capacidades, dispositivo y contexto reales, en vez de imponer la lógica interna del sistema. Esto reduce abandono, errores y necesidad de soporte.

4. Indica al menos ocho reglas de usabilidad y accesibilidad.

> 1. **Jerarquía clara:** usar un título principal y encabezados en orden lógico; no elegir niveles solo por su tamaño visual.
> 2. **Navegación consistente:** mantener etiquetas, posición y comportamiento del menú en todas las vistas.
> 3. **Teclado completo:** todas las acciones deben ser alcanzables con Tab, activables con teclado y sin trampas de foco.
> 4. **Foco visible:** cada enlace, botón y control necesita un indicador de foco con contraste suficiente; nunca eliminar el contorno sin reemplazo.
> 5. **Contraste y legibilidad:** usar combinaciones distinguibles, tipografía de tamaño adecuado, renglones razonables y espacio suficiente.
> 6. **Alternativas textuales:** escribir `alt` útil para imágenes informativas y `alt=""` para imágenes decorativas.
> 7. **Controles con nombre:** asociar `label` con campos y dar nombres accesibles a botones de solo ícono.
> 8. **No depender del color:** errores, estados y gráficas deben incluir texto, patrón o símbolo además del color.

9. **Mensajes accionables:** explicar qué ocurrió, dónde está el problema y cómo resolverlo; conservar la entrada válida.
10. **Objetivos táctiles cómodos:** enlaces y botones necesitan superficie y separación suficientes para reducir toques accidentales.
11. **Diseño adaptable:** evitar desplazamiento horizontal a anchos comunes y permitir zoom sin pérdida de contenido.
12. **Movimiento controlable:** respetar `prefers-reduced-motion`, evitar parpadeos y permitir pausar contenido animado.
13. **Lenguaje comprensible:** frases breves, instrucciones directas y términos consistentes; expandir abreviaturas no familiares.
14. **HTML nativo primero:** preferir `button`, `a`, `input` y landmarks antes que recrearlos con `div` y ARIA.

5. ¿Cómo verificar estas reglas?

Se combinan revisiones manuales y herramientas: recorrer solo con teclado; probar zoom a 200 % y reflujo; inspeccionar contraste; escuchar la página con lector de pantalla; revisar etiquetas y texto alternativo; probar mensajes de error; ejecutar validadores y auditorías automáticas; y realizar tareas reales con usuarios. La automatización detecta ciertos patrones, pero no puede decidir por sí sola si un texto alternativo es útil o si un flujo resulta comprensible.

Parte VIII. Análisis de un sitio web conocido

1. Analiza la usabilidad y accesibilidad de un sitio reconocido.

> **Sitio elegido: Wikipedia en español.** Es una enciclopedia colaborativa orientada a consultar, relacionar y editar conocimiento. El análisis es heurístico y conceptual; las características pueden variar por versión, sesión, dispositivo o artículo.

Aspecto solicitado	Observación
Menú de navegación	La búsqueda, el índice y los enlaces internos facilitan localizar contenido, aunque la cantidad de opciones puede ser alta.
Colores	Predominan fondos claros, texto oscuro y enlaces diferenciados; la sobriedad favorece la lectura.
Tipografía	Es legible y mantiene jerarquías claras entre título, subtítulos, cuerpo y referencias.
Organización	Los artículos usan introducción, índice, secciones, referencias e información complementaria.
Formularios	La búsqueda es directa; los formularios de edición son más complejos y requieren aprendizaje.
Botones	Las acciones principales son reconocibles, aunque algunos controles secundarios compiten por atención.
Mensajes	Los avisos informan estado editorial y mantenimiento, pero varios avisos simultáneos aumentan densidad.
Imágenes	Incluyen leyendas y contexto; la calidad del texto alternativo depende de las contribuciones.
Adaptación móvil	El contenido se reorganiza para pantallas estrechas, pero ciertas tablas todavía necesitan desplazamiento lateral.

Cinco aspectos positivos

1. **Propósito evidente:** el título del artículo, la entrada de búsqueda y el contenido principal permiten reconocer rápidamente la tarea.
2. **Estructura extensa pero navegable:** encabezados, índice y enlaces internos ayudan a saltar entre secciones sin recorrer todo el texto.
3. **Convenciones familiares:** los enlaces se distinguen, la búsqueda ocupa un lugar predecible y las referencias siguen patrones consistentes.
4. **Contenido alternativo y semántico:** muchas imágenes poseen descripciones y el contenido enciclopédico suele organizarse con encabezados, listas, tablas y leyendas.
5. **Adaptación y preferencias:** el sitio ofrece presentación para distintos anchos y permite ajustar aspectos de lectura; su contenido textual funciona aun con capacidades limitadas.

Cinco oportunidades de mejora

1. **Densidad cognitiva:** artículos largos, referencias, avisos y controles simultáneos pueden abrumar. Podrían jerarquizarse mejor acciones secundarias.

2. **Tablas complejas:** algunas infoboxes o tablas creadas por la comunidad requieren desplazamiento lateral y pueden ser difíciles con ampliación o lector de pantalla.
3. **Consistencia del texto alternativo:** la calidad depende de cada contribución; convendría reforzar guías y controles editoriales para imágenes significativas.
4. **Lenguaje especializado:** ciertos artículos empiezan con definiciones densas. Resúmenes introductorios en lenguaje sencillo ayudarían a públicos no expertos.
5. **Carga de navegación por teclado:** la cantidad de enlaces puede volver largo el recorrido. Atajos, enlaces de salto visibles al foco y landmarks consistentes reducen esfuerzo.

2. ¿Qué puede adoptar el proyecto de empresa tecnológica?

> Puede adoptar títulos descriptivos, navegación predecible, enlaces internos, diseño centrado en lectura y contenido disponible sin interacciones complejas. Debe evitar páginas saturadas, limitar acciones secundarias, presentar tablas adaptables, escribir textos breves y probar el recorrido con teclado desde el primer prototipo.

Parte IX. CSS y CSS3

1. ¿Qué es CSS y para qué sirve?

> **CSS (Cascading Style Sheets)** es el lenguaje que define cómo se presenta un documento: colores, tipografía, espaciado, tamaño, bordes, fondos, distribución y adaptación. “Cascada” significa que varias declaraciones pueden competir y el navegador resuelve el valor final según origen, importancia, capa, especificidad y orden.
>
> Separar CSS de HTML permite reutilizar estilos, conservar coherencia y cambiar la identidad visual sin alterar el significado del contenido. Puede incorporarse en línea, dentro de `style` o mediante un archivo externo con `link`; para proyectos se prefiere el archivo externo por mantenimiento y caché.

2. ¿Qué significa CSS3?

> CSS3 se usó para nombrar la etapa en que CSS se organizó en módulos y sumó capacidades como media queries, transiciones, transformaciones, fondos múltiples, sombras, bordes redondeados, Flexbox y Grid. Actualmente los módulos evolucionan de forma independiente, por lo que en documentación técnica resulta más preciso hablar de CSS moderno y del nivel de cada módulo, no de una única versión monolítica.

3. ¿Qué beneficios aporta CSS moderno?

> - Diseños fluidos con Flexbox, Grid, funciones como `min()`, `max()` y `clamp()`.
> - Variables con propiedades personalizadas para colores, espacios y temas.
> - Adaptación por medio, tamaño, orientación y preferencias de usuario.
> - Interacciones visuales con transiciones y transformaciones sin sacrificar la estructura.
> - Mejor aislamiento mediante metodologías, capas de cascada y selectores funcionales.

Parte X. Regla CSS y selectores

1. ¿Cómo se compone una regla CSS?

```
.tarjeta {
 color: #173e54;
 padding: 1rem;
}
/* selector { propiedad: valor; propiedad: valor; } */
```

`.tarjeta` es el selector; cada pareja `propiedad: valor` es una declaración; las declaraciones se encierran entre llaves y terminan con punto y coma. En este caso se seleccionan elementos con la clase `tarjeta` y se les asignan color y espacio interior.

2. Explica los principales selectores con ejemplos.

Tipo	Ejemplo	Qué selecciona
Universal	`* { box-sizing: border-box; }`	Todos los elementos.
Tipo o etiqueta	`p { line-height: 1.6; }`	Todos los párrafos.
Clase	`.boton { … }`	Elementos cuya clase incluye `boton`.
Identificador	`#contacto { … }`	El elemento con ese `id` único.
Agrupación	`h1, h2, h3 { … }`	Cualquiera de los selectores separados por coma.
Atributo presente	`input[required] { … }`	Campos que contienen `required`.
Atributo exacto	`input[type="email"] { … }`	Campos cuyo tipo es exactamente email.
Palabra en atributo	`[class~="activa"] { … }`	Atributo con la palabra separada `activa`.
Prefijo	`a[href^="https"] { … }`	Enlaces cuyo destino inicia con `https`.
Sufijo	`a[href$=".pdf"] { … }`	Enlaces cuyo destino termina en `.pdf`.
Subcadena	`a[href*="docs"] { … }`	Enlaces cuyo destino contiene `docs`.
Descendiente	`nav a { … }`	Enlaces dentro de `nav`, a cualquier profundidad.
Hijo directo	`ul > li { … }`	`li` cuyo padre inmediato es `ul`.
Hermano adyacente	`h2 + p { … }`	Primer párrafo inmediatamente posterior a un `h2`.
Hermanos generales	`h2 ~ p { … }`	Párrafos hermanos posteriores al `h2`.
Pseudoclase de interacción	`a:hover`, `button:focus-visible`	Estado de puntero o foco visible.
Pseudoclase de formulario	`input:valid`, `input:invalid`	Campos según su validez.
Pseudoclase estructural	`li:first-child`, `tr:nth-child(even)`	Posición entre hermanos.
Pseudoclase funcional	`:not(.oculto)`, `:is(h2, h3)`	Exclusión o agrupación lógica.
Pseudoelemento	`p::first-letter`, `.dato::before`	Parte virtual o generada de un elemento.

3. ¿Cómo funciona la especificidad?

> Cuando dos reglas aplicables declaran la misma propiedad, se comparan, entre otros factores, la importancia y la especificidad. En forma simplificada: un identificador pesa más que una clase, atributo o pseudoclase; estos pesan más que una etiqueta o pseudoelemento. Si empatan, suele ganar la declaración posterior dentro del mismo contexto. Se recomienda trabajar con clases de baja especificidad, evitar `!important` como solución habitual y organizar la cascada para que sea predecible.

Parte XI. Propiedades, modelo de caja, Flexbox, responsive y frameworks

A. Propiedades CSS

1. ¿Qué propiedades se usan con mayor frecuencia?

Categoría	Propiedades	Finalidad
Tipografía	`font-family`, `font-size`, `font-weight`, `line-height`, `text-align`	Legibilidad y jerarquía textual.
Color y fondo	`color`, `background-color`, `background-image`	Identidad y contraste.
Tamaño	`width`, `max-width`, `min-height`, `aspect-ratio`	Dimensiones y límites fluidos.
Espaciado/borde	`margin`, `padding`, `border`, `border-radius`	Separación y contorno.
Distribución	`display`, `position`, `flex`, `grid-template-columns`, `gap`	Flujo y alineación.
Visibilidad	`overflow`, `opacity`, `visibility`	Recorte y presentación.
Interacción	`cursor`, `transition`, `transform`, `outline`	Retroalimentación visual y foco.

Conviene preferir unidades relativas: `rem` para tamaños ligados a la raíz, `em` para el contexto, porcentajes para proporciones y unidades de viewport con límites. Los píxeles siguen siendo útiles para bordes o detalles, pero una interfaz no debe depender de dimensiones rígidas.

B. Modelo de caja

2. ¿Qué es el modelo de caja?

Cada elemento se representa como una caja compuesta, desde adentro hacia afuera, por **contenido**, **padding**, **borde** y **margin**. En el modelo inicial `content-box`, el ancho declarado solo mide el contenido; padding y borde se agregan. Con `border-box`, el ancho declarado incluye contenido, padding y borde, lo que vuelve más predecible la distribución.

```
*, *::before, *::after { box-sizing: border-box; }

.tarjeta {
  width: 320px;      /* ancho total con border-box */
  padding: 24px;     /* espacio interior */
  border: 2px solid #00a7a5;
  margin: 16px;      /* espacio exterior */
}
```

Los márgenes verticales de ciertos elementos en flujo normal pueden colapsar. El `outline` se dibuja fuera de la caja sin ocupar espacio y es útil para el foco; no debe confundirse con el borde.

C. Flexbox

3. ¿Qué es Flexbox y cómo se utiliza?

> Flexbox es un modelo unidimensional para distribuir elementos en una fila o columna. El contenedor usa `display: flex`; `flex-direction` define el eje principal; `justify-content` distribuye sobre ese eje; `align-items` alinea sobre el eje transversal; `gap` separa; y `flex-wrap` permite saltos de línea. En los hijos, `flex-grow`, `flex-shrink` y `flex-basis` controlan crecimiento, reducción y base.
>
> ```
> .servicios {
>  display: flex;
>  flex-wrap: wrap;
>  gap: 1rem;
>  justify-content: center;
>  align-items: stretch;
> }
>
> .servicio {
>  flex: 1 1 16rem;
>  max-width: 24rem;
> }
> ```
>
> Es apropiado para navegación, grupos de botones, tarjetas y alineación de componentes. Para distribución bidimensional de filas y columnas, CSS Grid suele ser más adecuado. Ambos pueden combinarse.

D. Diseño responsivo, Mobile First y media queries

4. ¿Qué es diseño web responsivo?

> Es un enfoque para que contenido, controles y distribución se adapten a diferentes tamaños, orientaciones y capacidades sin crear una página separada para cada dispositivo. Se apoya en cuadrículas flexibles, imágenes adaptables, unidades relativas, límites de ancho, contenido prioritario y consultas de medios. “Responsivo” no significa únicamente reducir elementos: implica conservar comprensión, orden, operabilidad y legibilidad.
>
> Es importante porque las personas acceden desde teléfonos, tabletas, computadoras portátiles, monitores amplios y equipos con orientación o zoom diferentes. Una adaptación correcta evita texto diminuto, elementos superpuestos, controles difíciles de tocar y desplazamiento horizontal innecesario.

5. ¿Qué significa Mobile First?

> Mobile First consiste en definir primero la experiencia esencial para pantallas estrechas y después enriquecerla cuando existe espacio. Ayuda a priorizar contenido y rendimiento. En CSS, los estilos base resuelven móvil y las media queries con `min-width` agregan columnas o espacios progresivamente.

```
.contenedor { width: min(100% - 2rem, 72rem); margin-inline: auto; }
.tarjetas { display: grid; grid-template-columns: 1fr; gap: 1rem; }
img { max-width: 100%; height: auto; }

@media (min-width: 48rem) {
  .tarjetas { grid-template-columns: repeat(2, 1fr); }
}

@media (min-width: 64rem) {
  .tarjetas { grid-template-columns: repeat(3, 1fr); }
}

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { scroll-behavior: auto; transition: none; }
}
```

6. ¿Qué son las media queries?

Son reglas condicionales que aplican estilos si el medio cumple una condición, por ejemplo ancho, orientación, puntero, contraste o preferencia de movimiento. Un **breakpoint** es el punto en el que se decide cambiar la composición porque el contenido ya no cabe o deja de ser cómodo; debe elegirse por las necesidades del diseño, no por intentar cubrir cada modelo de dispositivo. También se deben evitar cambios de orden visual que contradigan el orden del DOM.

```
@media (max-width: 768px) {
 .contenedor {
   width: 100%;
 }

 .acciones {
   flex-direction: column;
 }
}
```

Este ejemplo hace fluido el contenedor y apila las acciones cuando el ancho disponible es de 768 píxeles o menos.

7. Describe pruebas conceptuales para escritorio, tableta y móvil.

Vista	Comportamiento esperado	Pruebas
Escritorio □ 1024 px	Navegación horizontal; hero con texto y recurso visual; tres tarjetas; formulario con campos relacionados en columnas cuando resulte claro.	Leer sin líneas excesivamente largas; foco visible; zoom 200 %; no tapar anclas con cabecera fija; validar uso con ratón y teclado.
Tableta 768–1023 px	Dos tarjetas por fila; navegación compacta; espacios moderados; controles táctiles cómodos.	Orientación vertical y horizontal; sin superposición; tablas con estrategia de reflujo o scroll identificado; imágenes sin deformación.

Vista	Comportamiento esperado	Pruebas
Móvil 320–767 px	Una columna; menú expandible con nombre y estado accesibles; botones amplios; campos a ancho completo; contenido prioritario primero.	320 px y 375/390 px; zoom; teclado virtual; mensajes de error visibles; ausencia de scroll horizontal; objetivos táctiles separados.

Además se prueban contenido largo, traducciones, imágenes ausentes, conexión lenta, preferencia de movimiento reducido y navegación con lector de pantalla. Las categorías son referencias, no una lista cerrada de dispositivos.

E. Investigación comparativa de frameworks CSS

8. ¿Qué es un framework CSS y cuándo conviene utilizarlo?

Es un conjunto reutilizable de estilos, convenciones y, en algunos casos, componentes interactivos que acelera la construcción de interfaces. Conviene cuando el equipo necesita consistencia, prototipos rápidos, cuadrícula probada y componentes comunes. No reemplaza HTML semántico, decisiones de accesibilidad ni CSS: debe evaluarse el costo de descarga, la personalización, la dependencia y el conocimiento del equipo.

Bootstrap

Qué es: framework de enfoque Mobile First con sistema de cuadrícula, utilidades, estilos base y componentes. Algunos componentes interactivos requieren su JavaScript.

Usos: paneles administrativos, sitios corporativos, prototipos y aplicaciones con patrones convencionales.

Características: contenedores, filas y 12 columnas; breakpoints; Sass y variables CSS; utilidades; componentes accesibles como punto de partida.

Ventajas: documentación amplia, desarrollo veloz, consistencia, ecosistema conocido y componentes listos.

Desventajas: apariencia reconocible si no se personaliza, CSS/JS no utilizado si se importa completo y marcado con varias clases.

Componentes: navbar, cards, modal, accordion, alerts, forms, buttons, carousel, offcanvas y dropdown.

Grid: `.container` → `.row` → `.col-*`; por ejemplo `.col-12 .col-md-6 .col-lg-4`.

Documentación oficial: https://getbootstrap.com/docs/

Tailwind CSS

Qué es: framework “utility-first” que ofrece clases pequeñas para componer diseños directamente en el marcado y genera CSS según las clases utilizadas.

Usos: sistemas de diseño personalizados, aplicaciones de producto y equipos que prefieren composición por utilidades.

Características: utilidades responsivas y de estado, tokens de diseño, configuración y optimización del resultado.

Ventajas: gran control visual, consistencia por escala, poca necesidad de inventar nombres de clases y CSS de producción ajustado al uso con una configuración correcta.

Desventajas: HTML con cadenas extensas de clases, curva de aprendizaje, necesidad de abstraer patrones repetidos y pocos componentes completos en el núcleo.

Componentes: el núcleo aporta utilidades, no un catálogo equivalente a Bootstrap; pueden construirse componentes o utilizar bibliotecas del ecosistema.

Grid: utiliza utilidades CSS Grid/Flexbox, como `grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4`.

Documentación oficial: https://tailwindcss.com/docs

Foundation

Qué es: familia de frameworks responsivos de ZURB; Foundation for Sites está orientado a sitios y aplicaciones personalizables.

Usos: productos que requieren una base flexible, prototipos responsivos y proyectos que desean configurar componentes mediante Sass.

Características: cuadrícula XY Grid, utilidades responsivas, componentes de navegación, formularios y complementos JavaScript.

Ventajas: alto nivel de personalización, cuadrícula potente, enfoque responsivo y conjunto amplio de componentes.

Desventajas: comunidad y disponibilidad de ejemplos menores que Bootstrap, curva de configuración y dependencia de JavaScript en widgets interactivos.

Componentes: accordion, callout, dropdown, menu, off-canvas, orbit, reveal, tabs, tooltip y formularios.

Grid: XY Grid emplea contenedores `grid-x` / `grid-y` e hijos `cell` con tamaños por breakpoint.

Documentación oficial: https://get.foundation/sites/docs/

9. Tabla comparativa y elección para el proyecto.

Criterio	Bootstrap	Tailwind CSS	Foundation
Enfoque	Componentes + utilidades	Utilidades para composición	Base configurable + componentes
Inicio rápido	Muy alto	Alto, con proceso de construcción recomendado	Medio
Personalización visual	Variables CSS/Sass y temas	Muy alta mediante tokens y utilidades	Alta mediante Sass y configuración
Componentes incluidos	Amplio catálogo	No es el objetivo principal del núcleo	Catálogo amplio
Cuadrícula	12 columnas con breakpoints	Utilidades nativas de Grid/Flex	XY Grid
JavaScript propio	Para componentes interactivos	No requerido por las utilidades base	Para componentes interactivos

Criterio	Bootstrap	Tailwind CSS	Foundation
Riesgo principal	Diseño genérico/peso no usado	Marcado cargado de clases	Menor ecosistema y aprendizaje
Mejor escenario	Sitio corporativo rápido y consistente	Producto con diseño muy personalizado	Proyecto configurable con cuadrícula flexible

Elección razonada. Para el proyecto académico de una empresa tecnológica se elige **Bootstrap** porque permite construir rápidamente una navegación responsiva, tarjetas de servicios, botones y formularios consistentes. La cuadrícula reduce el CSS de distribución y sus clases por breakpoint facilitan demostrar Mobile First. La elección no elimina la responsabilidad de personalizar la identidad, cargar solo lo necesario, escribir HTML semántico y probar accesibilidad.

Reflexión final: 15 respuestas sobre el proyecto

Las respuestas corresponden al proyecto **Nexo Digital**, una página para una empresa de tecnología desarrollada con HTML5, CSS3, JavaScript y Bootstrap.

1. ¿Cuál es la función de HTML5 dentro de una página web?

> HTML5 organiza y describe el contenido de la página: títulos, párrafos, imágenes, enlaces, listas, secciones y formularios. En Nexo Digital funciona como el esqueleto semántico sobre el cual se aplican los estilos y las interacciones.

2. ¿Qué relación existe entre HTML y el DOM?

> HTML es el código fuente que describe el documento, mientras que el DOM es el árbol de objetos que el navegador construye al interpretarlo. JavaScript consulta y modifica ese árbol para actualizar la interfaz sin cambiar directamente el archivo guardado.

3. ¿Por qué es importante utilizar etiquetas semánticas?

> Porque comunican la función de cada región. Elementos como `header`, `nav`, `main`, `article`, `aside` y `footer` facilitan el mantenimiento, ayudan a buscadores y permiten que las tecnologías de asistencia comprendan mejor la estructura.

4. ¿Qué función cumplen los metadatos?

> Describen el documento para el navegador y otros servicios. En el proyecto definen la codificación UTF-8, el ancho adaptable mediante viewport, la descripción, el autor, el color del navegador y el título mostrado en la pestaña.

5. ¿Qué ventajas ofrecen los formularios HTML5?

> Ofrecen controles adecuados para correo, teléfono y fecha, además de validaciones declarativas como `required`, `minlength`, `maxlength` y `pattern`. Esto proporciona retroalimentación inmediata y reduce código, aunque un sistema real también debe validar en el servidor.

6. ¿Cuál es la función de CSS?

> CSS controla la presentación visual: colores, tipografías, tamaños, espacios, bordes, fondos, distribución y adaptación. En Nexo Digital crea la identidad gráfica y complementa los componentes de Bootstrap.

7. ¿Qué diferencia existe entre HTML y CSS?

> HTML estructura y da significado al contenido; CSS define cómo se presenta. Separarlos permite cambiar el diseño sin perder la organización semántica y facilita reutilizar estilos en distintos elementos.

8. ¿Qué ventajas ofrece CSS3?

> CSS moderno aporta Flexbox, Grid, media queries, variables, gradientes, sombras, transiciones, transformaciones y funciones fluidas. Estas herramientas permiten interfaces atractivas y adaptables sin depender de imágenes o scripts para cada efecto.

9. ¿Qué significa diseño responsivo?

> Significa que el contenido y la distribución se ajustan al espacio disponible sin perder legibilidad ni provocar desplazamiento horizontal innecesario. El mismo sitio puede utilizarse correctamente en computadora, tableta y teléfono.

10. ¿Qué es Mobile First?

> Es una estrategia que diseña primero la experiencia para pantallas estrechas y después agrega mejoras para espacios mayores. Ayuda a priorizar contenido, rendimiento y controles táctiles desde el comienzo.

11. ¿Qué función cumplen las media queries?

> Aplican reglas CSS cuando se cumple una condición, como un ancho máximo, orientación o preferencia de movimiento. El proyecto las utiliza para reorganizar encabezados, estadísticas, botones, tarjetas y pie de página en pantallas reducidas.

12. ¿Qué ventajas tiene utilizar un framework CSS?

> Acelera el desarrollo con una cuadrícula, utilidades y componentes consistentes ya preparados. También ofrece documentación y patrones probados, pero todavía es necesario personalizar, escribir HTML semántico y verificar accesibilidad.

13. ¿Qué framework utilizaste en tu proyecto?

> Utilicé **Bootstrap 5.3.8**. Se aplicó en la barra de navegación, sistema de columnas, tarjetas de servicios, botones, formulario y utilidades responsivas. Los comentarios del código identifican los elementos pertenecientes al framework.

14. ¿Qué problemas encontraste al adaptar tu página a diferentes tamaños de pantalla?

> Los principales retos fueron evitar desbordamientos, mantener una jerarquía visual clara, convertir el menú horizontal en uno colapsable, apilar campos y botones, ajustar las imágenes y conservar controles táctiles cómodos. Se resolvieron con contenedores fluidos, el grid de Bootstrap, Flexbox y media queries.

15. ¿Qué mejorarías de tu proyecto?

> En una siguiente versión conectaría el formulario con un servicio seguro, añadiría una política de privacidad real, optimizaría aún más los recursos, realizaría pruebas con usuarios y automatizaría auditorías de accesibilidad, HTML y rendimiento.

Conclusión general. La actividad confirmó que HTML5, CSS3, usabilidad, accesibilidad y diseño responsivo funcionan como un solo sistema. Bootstrap acelera la construcción, mientras que las decisiones sobre semántica, contenido y experiencia continúan siendo responsabilidad del desarrollador.

Bibliografía

1. MDN Web Docs. “HTML: Lenguaje de etiquetas de hipertexto”. https://developer.mozilla.org/es/docs/Web/HTML
2. MDN Web Docs. “Introducción al DOM”. https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model/Introduction
3. MDN Web Docs. “CSS”. https://developer.mozilla.org/es/docs/Web/CSS
4. MDN Web Docs. “Diseño web responsivo”. https://developer.mozilla.org/es/docs/Learn_web_development/Core/CSS_layout/Responsive_Design
5. WHATWG. “HTML Living Standard”. https://html.spec.whatwg.org/
6. World Wide Web Consortium (W3C), Web Accessibility Initiative. “WCAG 2 Overview”. https://www.w3.org/WAI/standards-guidelines/wcag/
7. World Wide Web Consortium (W3C). “CSS Snapshot”. https://www.w3.org/TR/CSS/
8. Bootstrap. “Documentation”. https://getbootstrap.com/docs/
9. Tailwind CSS. “Documentation”. https://tailwindcss.com/docs
0. Foundation. “Foundation for Sites Documentation”. https://get.foundation/sites/docs/

> **Nota de elaboración:** el contenido consultado en las fuentes oficiales fue comprendido, sintetizado y parafraseado con redacción original para fines académicos; no se reprodujeron fragmentos extensos de las fuentes. Los ejemplos de código y el análisis fueron elaborados específicamente para esta actividad.

Lista final de verificación

- □ HTML5 y estructura mínima
- □ DOM con diagrama
- □ Etiquetas estructurales
- □ Presentación con ejemplos
- □ Metadatos
- □ Formulario y validación
- □ Usabilidad y accesibilidad
- □ 14 reglas prácticas
- □ Análisis: 5 positivos y 5 mejoras
- □ CSS y CSS3
- □ Anatomía de regla CSS
- □ Selectores y ejemplos
- □ Propiedades y caja
- □ Flexbox
- □ Responsive y Mobile First
- □ Media queries y pruebas
- □ Tres frameworks y tabla
- □ 15 reflexiones
- □ Bibliografía oficial